

NaCo 21/22 assignment report, group 16

Luuk van Beusichem s2642395, Sam Chaman Zibai s1678876, Sam Vermeulen s2659352

Leiden Institute of Advanced Computer Science, The Netherlands

Abstract. Genetic algorithms are algorithms inspired by the way in which natural selection takes place. We present our own genetic algorithm and explain in detail how it works. We give an explanation of genetic algorithms and the one that we have written by presenting some pseudo-code. Our genetic algorithms performed better than we expected, for all of the mutations except the bitflip mutation. We also look at genetic algorithms in the chemistry field and see that it outperforms the usual method that was used for the task in question.

We also made a Cellular Automata(CA), we use this CA to solve the inverting problem. We have a C^t that we give to our GA along with some other variables. This in turn gives a C^0 that we use our CA on to get a new C^t . Then we use two different methods to determine how good the answer we generated is, which is expressed in similarity. There is also a small section dedicated to the usage of CA's in the field of architecture. Lastly we discuss the use fullness of GA in the problem we tried to solve.

1 Introduction

In this paper we will give a introduction to genetic algorithms as well as a description of the one we have implemented in python. We will explain the core concepts of a genetic algorithm while giving various examples on each topic and how we approached these problems and tried to tackle them.

The operators of a genetic algorithm will be explained and we will discuss whether for each of the two variants of each operator one outperforms the other and if that is the case what the reason behind this performance difference may be.

We will look at a paper in which a genetic algorithm has been used to solve a problem that is not computer science related. We will look at how the genetic algorithm was applied and why the usage of a genetic algorithm was considered. We will then take a look at the results and see whether the approach was successful, what could have been done better and whether the use of a genetic algorithm was the right choice.

We then will apply our GA to a variant of the inverting problem with the help of a Cellular Automaton. First we will show how we implement a Cellular Automaton from scratch. Then we will explain in more detail what this inverting problem looks like.

We will try different combinations of operators on this inverting problem and will evaluate which combination of operators is the best for certain cases of the inverting problem.

Finally, we will look at a paper in which a Cellular Automaton has been used outside of the field of Computer Science. We will discuss whether the approach to use this Cellular Automaton was successful or not.

2 Algorithm Description

Genetic Algorithms (GA) are part of a bigger group of algorithms called evolutionary algorithms. GA use natural selection to find solutions for a given problem. A GA uses a population of possible solutions and all solutions are represented in a discrete way. The set of all current solutions at a certain point is called a generation. The starting generation is generated randomly. In order to generate new generations, the GA uses a fitness function to determine which instances of the population are good and which not. In general, the higher the fitness score of an instance, the more likely that this instance will be chosen for reproduction. This process of selecting which solutions to keep is aptly called selection.

After we selected a certain amount of parents that get to reproduce, We generate a new generation of solutions by generating new data using the parents' genetic code. We produce children from the parents until our population is big enough. This process of repeated crossover is called Recombination.

The last operator of the GA is called mutation. During the mutation phase we make slight changes to the instances of the population in order get more variety in the generation. If a new generation does not provide a sufficient solution and the maximum amount of generations has not been exceeded, the whole process of selection, recombination and mutation will be repeated. In the next subsections we will have a closer look on variants of selection, recombination and mutation.

2.1 Recombination

The first operator we discuss is recombination. Recombination can be done in various ways. One way is called one-point crossover [10]. This means that a point somewhere in the genetic code of the parents is randomly selected. The part of DNA up to that point is taken from one parent, while all the DNA at and after that point is taken from the other one.

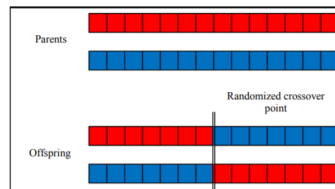


Fig. 1: Onepoint crossover [10]

A variant of this operator is called uniform crossover [10]. This variant entails that for each child, it loops over the genetic data of all parents bit by bit. For every bit it takes the value of the corresponding bit of one of the parents. The chance for each parent to pass on their bit is 1 divided by the amount of parents. Figure 1 and Figure 2 show that each variant of the recombination

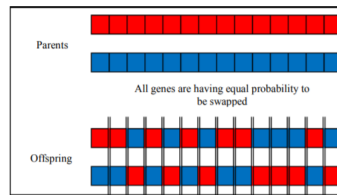


Fig. 2: uniform crossover [10]

operator leads to a totally different offspring. It seems to be beneficial to use uniform crossover in case you want to create more diverse children. In case you want some smaller changes in diversity, it might be beneficial to choose one-point crossover.

2.2 Mutation

One way of mutation is called swap mutation [9]. This type of mutation picks two data points within an individual and swaps their values. What these data points look like will be detailed in paragraph 2.4. Another mutation method is called bit flip mutation [2]. In our implementation of the operators we have named this operator bit string mutation, because we had already created another mutation operator called bit flip mutation. What our implementation of bit string mutation does is the following: It loops over all the bits of all children and toggles the bits between the possible values with a certain chance. The higher the chance, the more likely the value of the bit will change. The default value of this chance is 1 divided by the length of the child, so approximately one mutation happens in each of them. Swap mutation only changes the position of two data points and could therefore only change the value of two data points. Bit flip mutation could potentially change the value of all bits. It is better to choose swap mutations if your individuals have a high fitness score and you want to make minor changes in the next generation. The opposite holds for individuals with a low fitness score. You want to make bigger changes in order to get more variety in the population. It should be noted however, that a more chaotic mutation factor will, aside from increasing the chances of finding the global optimum, also decrease the speed at which all solutions move towards it.

2.3 Selection

The last search operator that we will look at is selection. We have implemented 2 variants for this. The first variant is called fitness proportional selection or rank selection [6]. The way we implemented this was by selecting the X solutions with the highest fitness values. This is not a necessarily rigid selection method, since it heavily encourages your algorithm to find a local optimum and stay there. That's why we implemented a more flexible selection operator as well. This method is called tournament selection [8]. For this type of selection we select a certain amount q random individuals from the population. The GA then compares the fitness of these q number of individuals and selects the best individual and puts it into the next generation. Both ways of selection have the property that the higher the fitness score of the individual, the more likely it is this individual will be selected. However, rank selection always has at least some decently performing individuals, where tournament selection actually gives the chance for lower fitness

individual to make the cut. Whether this is a good thing or a bad thing depends on what the experimenter wants to achieve. Although tournament selection makes for a more diverse gene pool, rank selection might be preferred if computing cost can't be high or that a global optimum is good enough.

2.4 Representation

Search points are generally represented as strings. These strings hold information. Each part of the string encodes certain information. For example if we use a binary representation, a 1 could mean that an item is present and a 0 could mean it is not. Instead of a binary representation, we could also use a discrete representation, where each cell can take more values instead of just 0 and 1. This representation is important because it defines what an algorithm can and can't do. More complex representations will allow for more complex behaviors, but an algorithm's genetic code shouldn't be more complex than it needs to be for storage and computation purposes.

In our implementation all our operators except bit-flip mutation can handle this discrete representation with more than just zeroes and ones in the genetic code. The reason the bit-flip mutation operator doesn't work on this discrete representation is that we have to define precisely what the value of the cell will become. In a binary representation you only have to deal with value 0 and 1 and we can freely toggle between them. This is in contrast to a ternary representation, where we have to deal with 3 values (0,1,2) and we cannot just toggle between 2 values. In order to handle multiple values, we have to define precisely what the value of the cell will become for every possible state of the cell. We implemented it the following way. 0 becomes 1, 1 becomes 2 and 2 becomes 0.

3 Implementation

To implement the algorithm we started from the code provided to us during the lecture and built our functions from there.

We created the selection, crossover and mutation methods that we thought would fit best with our objective:

1. **Rank selection:** We implemented rank selection because we have no need for complex strategies and this selection method is relatively simple to implement. On top of that, the way we calculate the highest ranking strings also runs in linear time, using numpy's *argpartition()* function.
2. **Tournament selection:** In order to create a more diverse gene pool, we implemented Tournament Selection. Our implementation divides all current solutions into k different groups. From each of these groups, the individual with the highest fitness is selected, leaving us with k parents to create children from. This selection method allows for solutions with slightly lower fitness scores to also produce offspring, decreasing the chances of getting stuck in a local maximum.
3. **Uniform Crossover:** We chose uniform crossover for similar reasons as Rank Selection. It's relatively straightforward to implement and doesn't get more complicated than we need, hopefully saving on computing power. In our program, to create the new strings (or children) from the parent strings, for each child individually we loop over the genetic data of all parents

bit by bit and copy over some data. Every bit, we choose one of the parents' bits to inherit, with each of the parents having a chance of $\frac{1}{\text{amount of parents}}$ to pass on their bit to the current child that's being created.

We wrote the function in such a way that any amount of parents between two and the maximum amount of strings in a population can create children.

4. **One-point Crossover:** We implemented One-point Crossover because it allows for the parents to pass on parts of their genes relatively intact. This could be beneficial when, for example, a big part of a parent is completely correct and it just needs some small adjustments through taking one part of another parent's code and slight mutations.
5. **Bitstring Mutation:** The first mutation algorithm we implemented was bitstring mutation. This mutation method has a certain chance to flip a single bit from one to zero and vice versa. The default value for this chance is $\frac{1}{\text{amount of bits in the genome}}$, but this value can also be specified when calling the function. The default value is set so that on average, one mutation will happen in each child.
6. **Bitflip Mutation:** Since bitstring mutation didn't always lead to the optimal solution and we wanted to experiment, we implemented bitflip mutation. This mutation method takes all the bits of all the children and flips them either from one to zero or from zero to one. This method performed worse than bitstring mutation, probably because the gene pool quickly becomes stale. This staleness is, of course, also in part because of our use of Rank selection. Using other, more randomized selection methods will probably work better in combination with this mutation method.
7. **Swap Mutation:** Finally we implemented swap mutation, because we wanted to increase the consistency of our algorithm being able to pass the test. Our implementation of swap mutation loops through *each* child and swaps the values of two of their genes. This means that the gene pool never really gets stale, which in turn causes continuous movement towards the optimum. We tested this method with many amounts of parents, variations of the amounts of strings in our gene pool and lengths of genomes and it always seems to find the optimum. Even with two parents, a maximum population of 5 and a genome that's 1000 bits long, it was able to find the optimum within 3000 steps.

To create our genetic algorithm from these parts, we put them all together, give it an objective and let it run. An important part of this algorithm is the fact that it can be easily adjusted without drastically changing the code. If a new function is added for example, it only needs to be put in the list of the GA's functions and get an if-statement in the GA's `__init__` function. Our GA comes with default values, like a population size of 100, but this value can be specified differently when calling the GA. An overlook of the specifiable values are in the table under here. The default values are in bold print.

Variable	Initialization or Call	Possible Values	Expected Input
Initialization Method	Initialization	Random Initialization Half Initialization	init_method == 'Random' init_method != 'Random'
Generation Method	Initialization	Random Generation Bitflip Generation	generation_method == 'Random' generation_method != 'Random'
Selection Method	Initialization	Rank Selection	selection_method == 'Rank'
Crossover Method	Initialization	Uniform Crossover	crossover_method == 'Uniform'
Mutation Method	Initialization	Bitstring Mutation Bitflip Mutation Swap Mutation	mutation_method == 'Bitstring' mutation_method == 'Bitflip' mutation_method == 'Swap'
Problem	Call	Any integer problem from the ioh library	ioh.problem.Integer
Population Size	Call	[2, MAXINT)	Any Number
Parent Amount	Call	[2, PopulationSize]	Any Number

Our default values for the population size is 100 and the default amount of strings that are picked to produce offspring is the population size divided by 10, which gets brought up to 2 in the case of a population smaller than 20.

For the population size and amount of parents, although we assume the input will be clean, we cast the sizes to integer types and adjust them to our boundaries to prevent any unwanted behavior.

When the GA gets initialized, it determines what functions it will use when it gets called. When the GA gets called it tries to reach the optimum value for the given problem in the following way:

Program Pseudocode

1. Determine the problem dimension and clean up the input.
2. Initialize the population
3. **While** the optimum has not been found and the maximum amount of problem evaluations hasn't been reached yet:
 - (a) Calculate the fitness of the population
 - (b) Find the fittest individuals using the selection method
 - (c) Create children from these individuals using the crossover method
 - (d) Mutate these children using the mutation method
 - (e) Submit the child with the highest fitness to the problem
 - (f) The children become the new population

We tested our algorithm manually using print statements and through calling *test_algorithm.py* from the command line.

4 Application of GA in practice

For this part of our paper we will look at the application of a GA in practice. The paper we will be discussing uses a genetic algorithm to optimize molecular geometry.

The simulation of molecular dynamics is a heavy task on the programs that want to do this job. The major limitations that these simulations have to deal with are the system size and simulation time. The usual method for doing this task is simulated annealing but this method doesn't perform as well when the number of atoms exceed a certain number. For this reason the paper looks at the usage of genetic algorithms and whether they outperform simulated annealing. [3]

The problem the paper aims to solve is what the ground structure of small atomic clusters look like. This is done by evaluating the energy and forces within the model potential. The most interesting state for these small atomic structures is the range between it being a molecule or bulk matter. This is doable for up to ten or twenty atoms but any more and the simulation will run painfully slow making it a simulated annealing an unusable solution to this problem. [3]

The genetic algorithm uses the atomic Cartesian coordinates as the starting population. The parent operation uses the geometries of two atoms out of the population to produce a child. The preferred atoms to be chosen are one with a lower energy which is done with the help of a Boltzmann distribution where the energy per atom divided by the median of all energies is used to distribute the chances of a atom being picked based on its energy. [3]

The Genetic algorithm uses two mutations and applies these to the population. The first mutation moves the atoms in a random direction a random amount of time for a random distance. The second mutation method moves an atom for a random amount of steps but makes sure its direction is towards a high energy atom. [3]

The genetic algorithm outperforms the simulated annealing and finds an answer in a reasonably fast amount of time for examples where simulated annealing cant even find an answer. During the first couple of generations the way in which the genetic algorithm approaches the problem is very similar to simulated annealing. The majority of the time is spent solving how the last atoms fit into the structure. [3]

Since the physical structure of the atoms can be translated into a binary number string the use of a genetic algorithm is not that far fetched. The mating procedure uses these numbers and based on our requirements mixes and matches such that the children will show the desired traits while having them be more optimized than their parents. Also since the ground state structures in the genetic algorithm are similar to the simulated annealing one we can conclude that the same factors that influence the simulated annealing also affect the genetic algorithm. [3]

Since the mating procedure is one of the most important factors in a genetic algorithm we would also try and add some other mating procedures to compare them to each other as to see if one outperforms the other since we already established that genetic algorithms outperform the default method that was used to tackle this problem.

5 Celular Automata

For this report we applied our GA to a problem with the help of a Cellular Automaton. A cellular automaton (CA) is a model of computation based on the principle of self-replicating [11]. In

CA, cells are positioned in an n-dimensional grid. For this report we will deal with a 1D grid. Each cell could take a finite number of values [11]. For this report we will deal with a scenario where a cell could take a 0 or 1 as value ($k=2$) and a scenario where it could take a 0,1 or 2 as value ($k=3$). At different iterations, we can update the state of the cells. This process of updating a cell is based on the state of the cell itself and the state of the neighboring cells and a transition rule [11]. The number of neighboring cells depends on the neighbourhood radius. A radius of 0 means, updating of the cell is based on the cell itself. A radius of 1 in a 1D grid means updates of the cell are based on the cell itself and its adjacent neighbors and so on. For this report we will deal with a radius (r) of 1. The transition rule tells us what the value of the cell will be, given a particular neighborhood. We represent the transition rule as a transition table. The size of the table = $k^{(2r+1)}$ [11]. The table below shows an example of a transition table for $k = 2$ and $r = 1$.

Neighborhood state	111	110	101	100	011	010	001	000
Output	0	1	1	1	1	1	1	0

The top row displays every possible configuration of the neighborhood. The bottom row displays the output for every configuration. This output depends on the given transition rule. In this case the transition rule = 126. If we convert 126 to a binary representation, we get 01111110 which is another way of writing the output row of the table. The boundary cells of the grid are special cells. They only have 1 adjacent cell and as we can see, the transition table of table 2 deals with neighborhoods that consist of 3 cells. In order to deal with these cells there are various approaches. For this report we will enclose the border of the grid with zero-cells. These zero-cells only play a role in updating the border cells and will never be updated. The initial state is called C^0 and after t iterations we ended up in C^t

Implementation of the CA

In order to implement our CA, we created a Class called CellularAutomata. Our `__init__` method, which serves as a constructor, is given a variable `rule_number`. This variable indicates which rule should be applied to our CA. We save the variable `rule_number` in our class attribute `rule`.

After we defined our `__init__` method, we define our `__call__` method. This method returns a C^t for a given C^0 after t timesteps. First, we initialized a list called `rules` that will represent our transition rules. Second, we determine whether we have a k with value 2 or 3. This value determines the number of transition rules as previously mentioned. Let's have a look at the scenario with $k = 2$. We transform the decimal rule number to an eight-bit binary number. We then append this binary number to our rules list bit by bit. We then have a list of size 8 that represents our transition rules. After our initialization of the rules list, we make a copy of the grid (`grid1`) and we continue working with this copy (`grid 2`).

We then enter a loop that will last t iterations. First, we add to our boundaries zero-cells, such that the border of our grid is enclosed by zero-cells. The values of these cells are fixed to zero. This all is done in order to handle the evaluation of the border cells which would have had only one neighbour otherwise. By adding these zero-cells we can evaluate the original border cells as normal cells. After we added the zero-cells we make a copy of `grid 2` and loop trough the

new version of the grid (grid 3) from the index of the left border of grid 1 to the index of the right border of grid 1. For each cell we check the previous, the current and next cell. There are 8 possible options for the neighbourhood and each possibility corresponds to one of our entries of the transition rules list. For example, if the previous cell = 1, the current cell = 1 and the next state = 1, the current cell will have the same value of the first entry of the rules list after this iteration in grid 3. For the combination 110 it will be the second entry and so on. After we evaluated every cell, we make a new copy of the grid and delete the two zero-cells we added at the beginning. If there have been t iterations, we return the current grid, otherwise we repeat this whole process.

The scenario with $k = 3$ looks similar. The only difference are the transition rules. For $k = 2$ we had 2^3 transition rules. For $k = 3$ we have $3^{(2+1)}$ transition rules. First, we transform the decimal rule number to a ternary number. The number of bits that represents this number is flexible. In order to represent the transition rules in a list, we make sure the list has 27 entries with value 0. We then loop through the rules list backwards by looking at the negative index. Every entry of the list gets the same value as the bit that has the same negative index in the ternary bitstring. After we finished the initialisation of the transition rules, we make a copy of the grid and follow the exact same procedure as described in the previous paragraph with a small side note that there are 27 possible options for the neighbourhood instead of 8.

6 Inverting problem

For this report we will tackle a version of the inverting problem. The inverting problem is a problem in the world of cellular automata. A regular inverting problem deals with the problem to calculate a transition rule that describes the transformation for a given C^0 to C^t after t iterations. An example of the inverse problem in real life is the Urban Growth Modelling [7]. The goal of this example is to find transition rules to simulate city growth. In order to find these transition rules for real life problems GA's are widely used. For our version of the inverse problem, we will use our GA implementation as well. The aim for our version of the inverse problem is to find a C^0 state that after t iterations will end up in C^t after following a certain transition rule. The transition rule, t and C^t are given and it is up to us to find C^0 .

Our GA will suggest various $C^{0'}$ as possible correct C^0 . In order to calculate the correctness of $C^{0'}$ we have to perform our CA on this suggestion. The t and transition rule are already given, so it is possible to calculate $C^{t'}$ by calling our CA with the associated arguments. After we have calculated $C^{t'}$ we need to calculate the similarity between this state and C^t . There are various approaches to do so. We have implemented 2.

The first method to calculate the similarity is calculating the Hamming distance [1]. The Hamming distance is a measure of the difference between two strings of equal length. In order to calculate the Hamming distance we loop through $C^{t'}$ and compare each bit with the bit at the same index of C^t . If they are equal the similarity increases. We end up with a similarity that is equal to the number of equal bits divided by the length of the string.

The second method to calculate the similarity is calculating the Levenshtein distance [5]. The Levenshtein distance is a measure to calculate the minimal number of single-character edits that is required to transform one string into the other. These edits are insertion of a character, deletion of a

character and substitution of a character. In order to calculate the Levenshtein distance, we use the Python-Levenshtein package. This package allows us to easily calculate the Levenshtein distance between $C^{t'}$ and C^t . The similarity is equal to (the dimension of the problem – the Levenshtein distance) divided by the dimension of the problem.

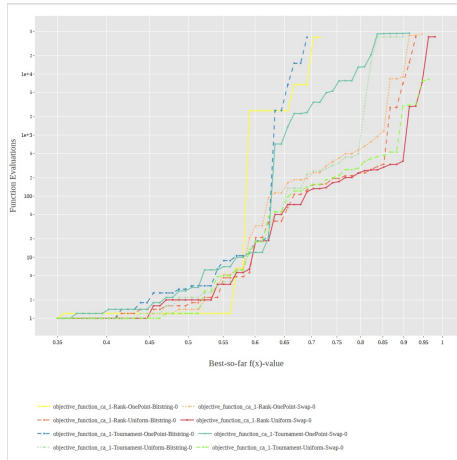


Fig. 3: problem 1, Hamming distance.

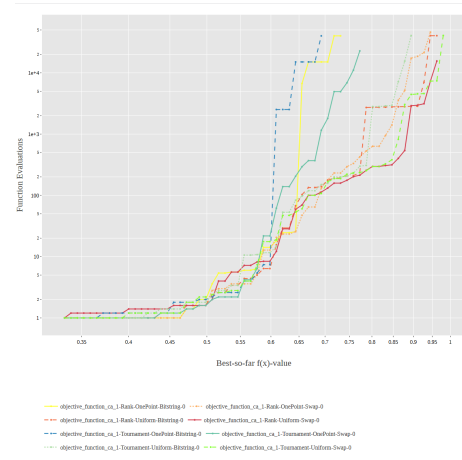


Fig. 4: problem 1, Levenshtein distance

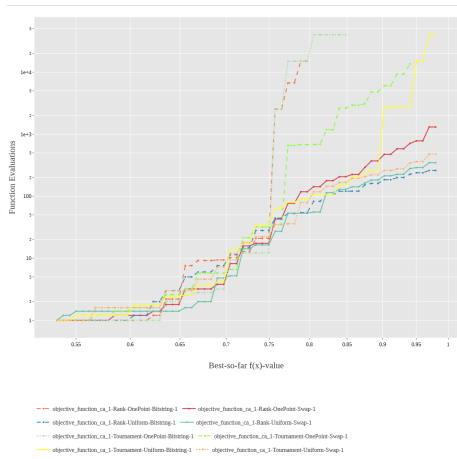


Fig. 5: problem 2, Hamming distance.

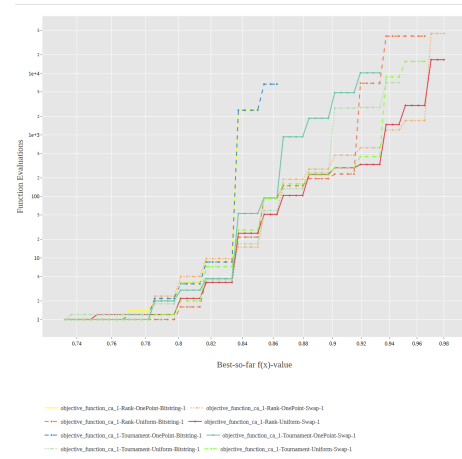


Fig. 6: problem 2, Levenshtein distance

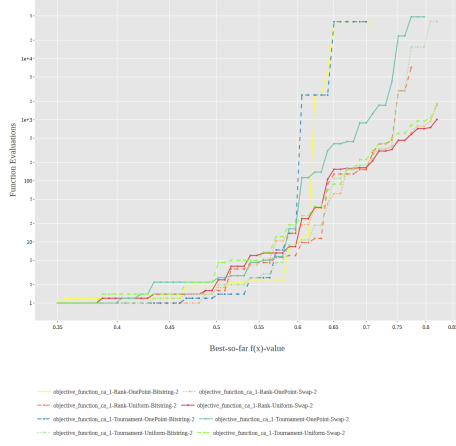


Fig. 7: problem 3, Hamming distance.

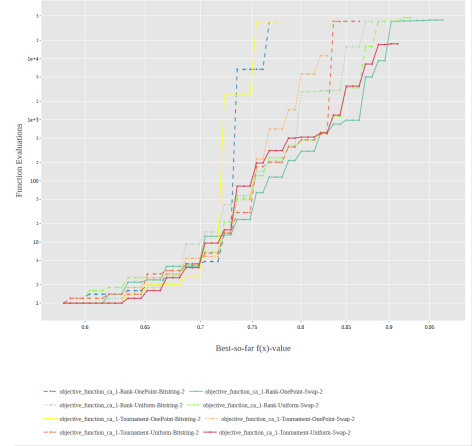


Fig. 8: problem 3, Levenshtein distance

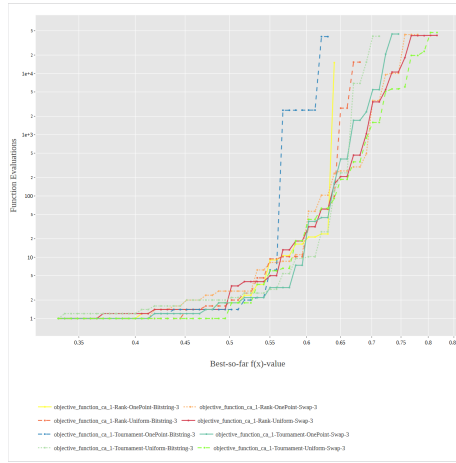


Fig. 9: problem 4, Hamming distance.

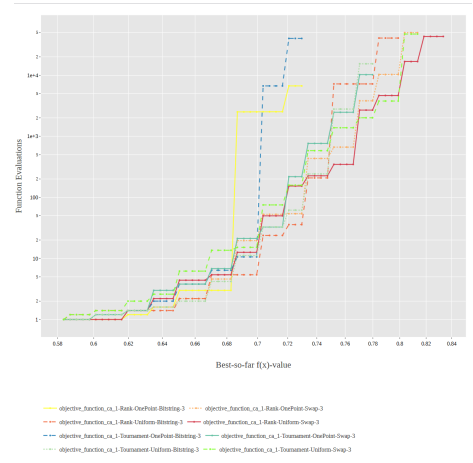


Fig. 10: problem 4, Levenshtein distance

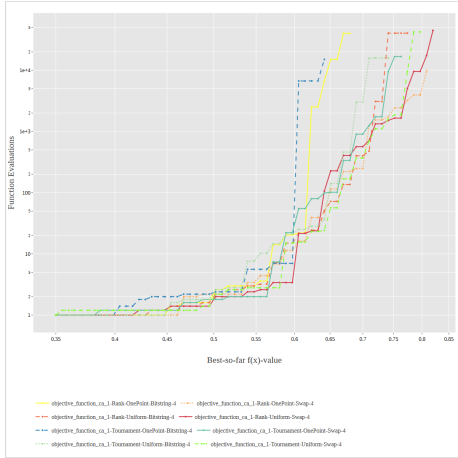


Fig. 11: problem 5, Hamming distance.

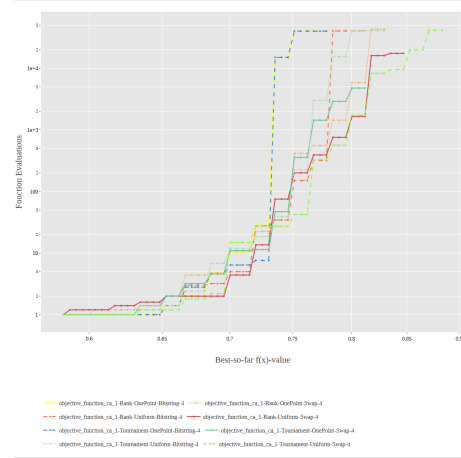


Fig. 12: problem 5, Levenshtein distance

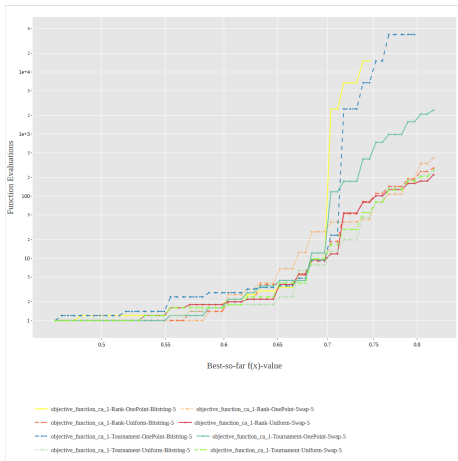


Fig. 13: problem 6, Hamming distance.

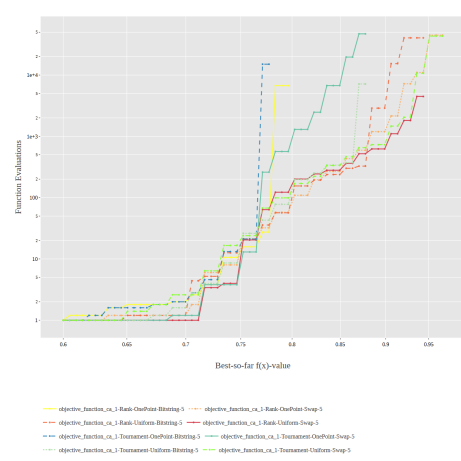


Fig. 14: problem 6, Levenshtein distance

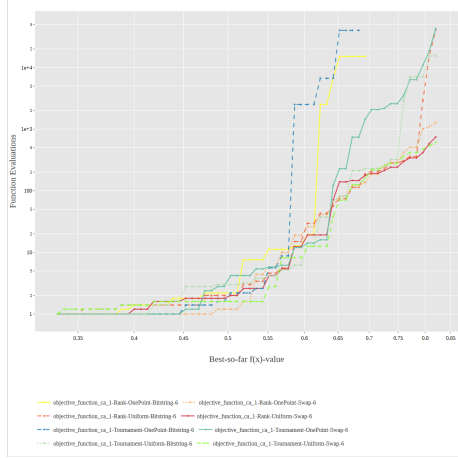


Fig. 15: problem 7, Hamming distance.

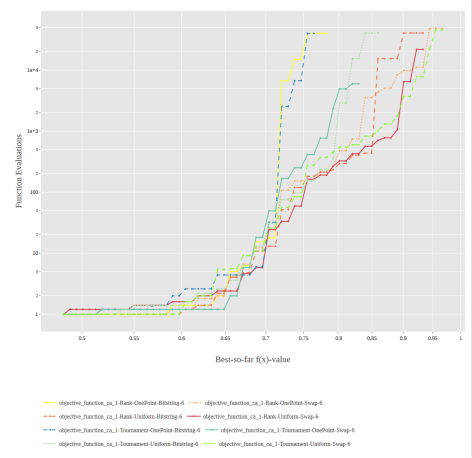


Fig. 16: problem 7, Levenshtein distance

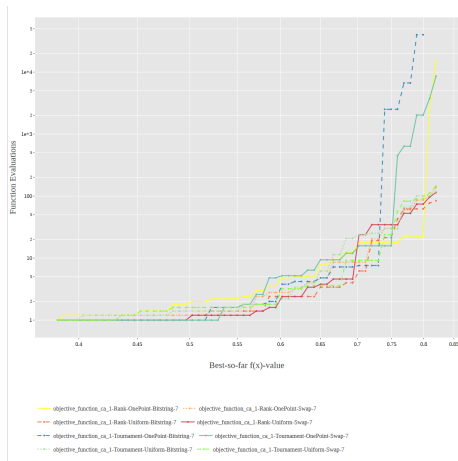


Fig. 17: problem 8, Hamming distance.

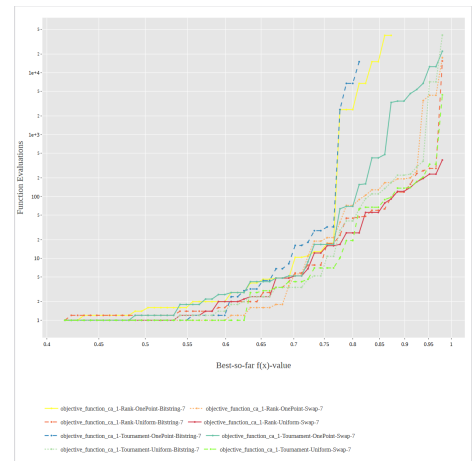


Fig. 18: problem 8, Levenshtein distance

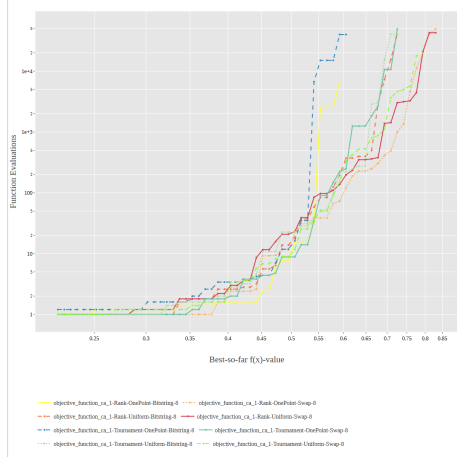


Fig. 19: problem 9, Hamming distance.

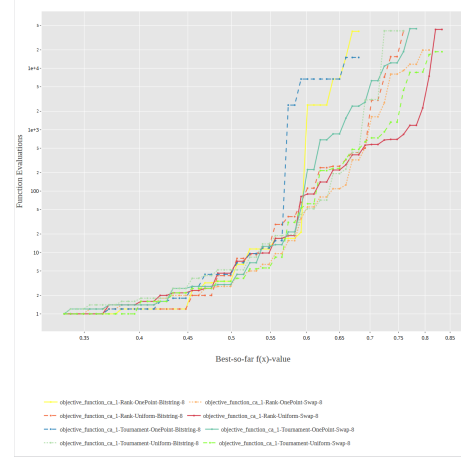


Fig. 20: problem 9, Levenshtein distance

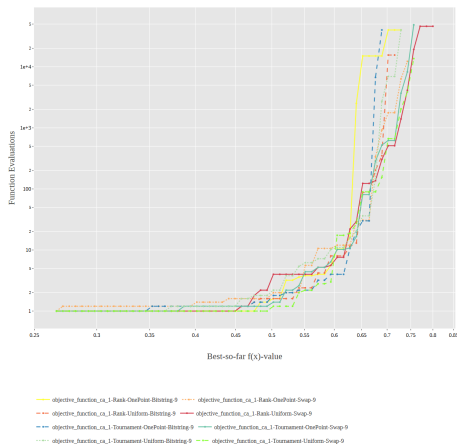


Fig. 21: problem 10, Hamming distance.

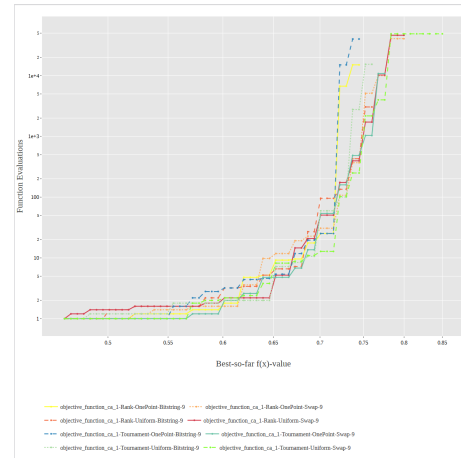


Fig. 22: problem 10, Levenshtein distance

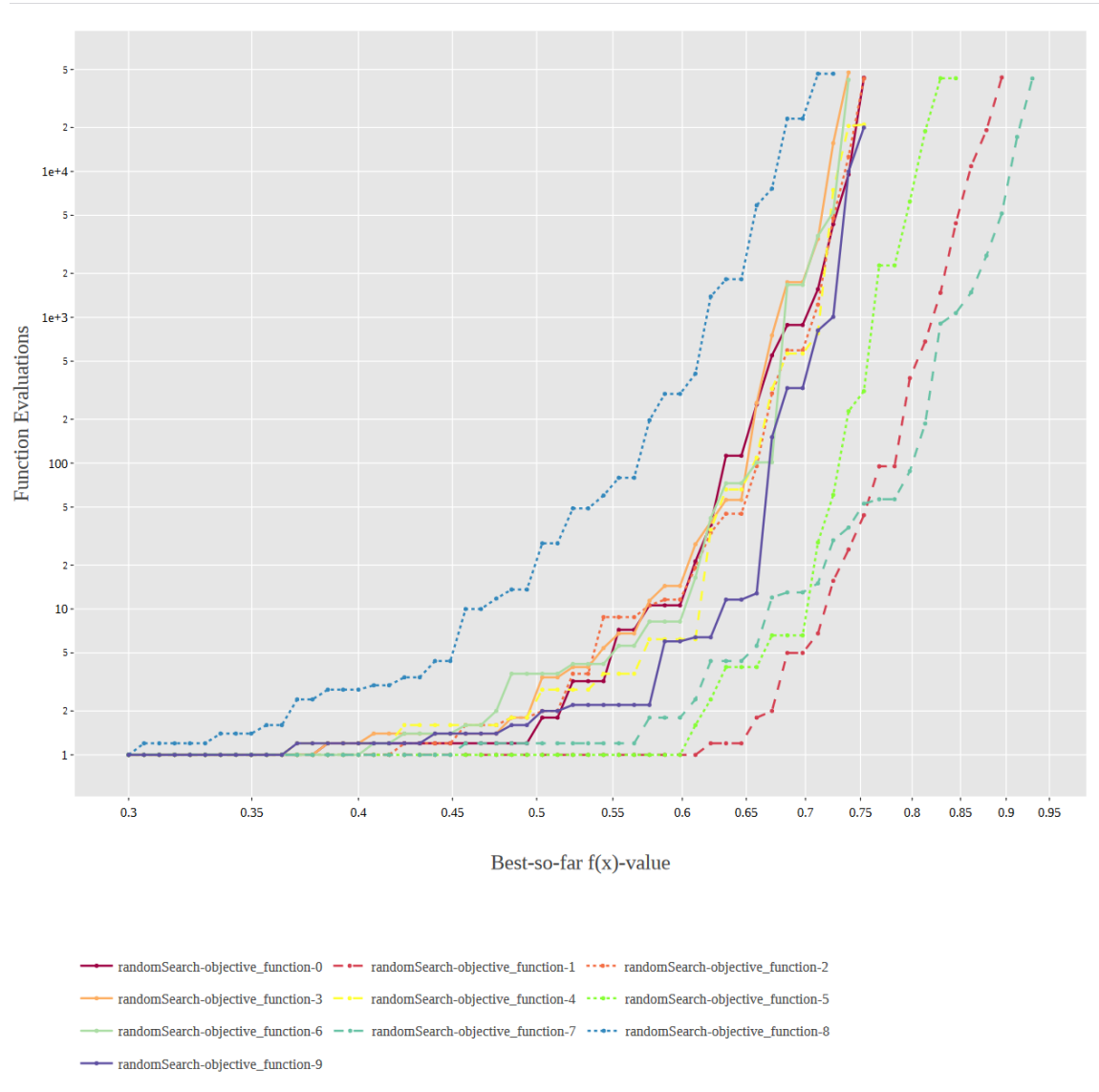


Fig. 23: All problems, Random search, Hamming distance.

7 Results analysis

We ran our experiments as follows: For every combination of selection, crossover and mutation operators, we ran 5 tests for both the Hamming distance and the Levenshtein distance as objective functions. We then took the mean of those 5 trials per configuration and plotted that in the figures. In total, with 2 options for all operators, this gives us $2 * 2 * 2 = 8$ operators, 2 objective functions, 10 problems and 5 trials for each configuration for a total of $8 * 2 * 5 * 10 = 800$ total trials, with $8 * 2 * 10 = 160$ different data sets. One for each configuration.

What's interesting about the first problem is that for every configuration of methods seems to perform the same on Hamming distance, as well as on Levenshtein distance. The two best configurations of methods in both cases are rank-uniform-swap and tournament-uniform-swap. The general trend throughout all graphs is that rank-onepoint-bitstring, tournament-onepoint-bitstring and tournament-onepoint-swap tend to perform poorly. We suspect this might be because using One-Point crossover leads to staling in the gene pool, slowing the movement towards the maximum. The only notable exception to this is rank-onepoint-swap.

For each problem, we have found what we think is the best configuration:

1. Rank-uniform-swap using Hamming distance and tournament-uniform-swap using Levenshtein distance
2. Rank-uniform-swap using Levenshtein distance
3. Rank-onepoint-swap using Levenshtein distance
4. Rank-uniform-swap using Levenshtein distance
5. Tournament-uniform-swap using Levenshtein distance
6. Tournament-uniform-swap using Levenshtein distance
7. Tournament-uniform-swap using Levenshtein distance
8. Rank-uniform-swap using Levenshtein distance
9. Tournament-uniform-swap using Levenshtein distance
10. Tournament-uniform-swap using Levenshtein distance

As you can see, all of the well-performing algorithms use uniform crossover. They also arguably all use swap mutation, while rank and tournament selection seem about equally distributed. It should also be noted that Levenshtein distance almost always scores higher than Hamming distance. Uniform crossover probably performs better than one-point crossover, because uniform crossover doesn't stale the gene pool nearly as much as one-point crossover does, resulting in faster movement towards the maximum. The reason that swap mutation performs better than bitstring is probably because it mutates two bits at the same time, instead of a mean of 1 per instance. This means that mutations are quicker and more radical, leading to more variety and possibly better solutions. The reason that rank and tournament selection seem to perform quite similarly is perhaps because of the low complexity of the problem. Solutions that pull the problem

state out of a local maximum often need to be radically different, which tournament selection probably won't help achieve any more than rank selection in this case. For example in the first problem, our algorithm always got stuck on some solution with many ones, while the actual solution was 59 zeroes and a one in the middle. We think that the reason Levenshtein distance usually scores better is perhaps because the evaluation method just produces higher values in general. Because our algorithm only cares about being as close to 1 as possible, we didn't pay the value distributions any attention.

8 Application of CA in practice

Cellular Automata's(CA) are mainly used in computer science, but this doesn't mean that other fields of work do not make use of them. The concept is simple, there are some base rules and dimensions which you define and want your cellular automata to use and the CA does the rest of the work. The important part is defining the right rules and dimensions for your problem and choosing ones that make sense with what you want your CA to achieve.

The topic for which we cover the use of CA's is architectural design processes. CA's produce patterns which have good spatial design and the unknown factor results in some interesting patterns that can help bolster the imagination of architects. Even though the use of CA's in architecture isn't uncommon, there aren't a lot of good documentations on the usage and application of them. This can be attributed to the usage being mainly focused on the results and not the way in which it is done. The paper we have reviewed tries to combat this by giving CA's literally attention in the field of architecture and explaining in depth the way in which it was used.

The usage of CA's can be attributed to two simple facts. The foundation of a CA is comprised of a few basic rules which means that implementing a CA is not as difficult as it may seem. The other fact is that even though CA's operate on fairly simple rules, the results they produce can be very complex. This means that the difficulty to result reward is pretty high for such a simple method, which makes it quite attractive for architectural problems.

The particular paper we reviewed did a case study on the use of CA's for designing hotel residences for scientists/engineers at an astronomical observatory. It is here that the paper tries to differ itself from previous works regarding CA in architecture by explaining the methods used and giving a clear description of their methods. [4]

The first attempt tried to apply rules for lightning, scale, structure, etc. but having too many rules made it so that the complexity of the problem even for a small design would be way too high and getting meaningful results would not be feasible. The next stage of the experiment focused itself on the general shape of the architectural design. The rules for this particular CA were that an alive cell with less than X_1 or more than X_2 surrounding alive cells would die, and a dead cell with more or equal to Y_1 or less and equal to Y_2 alive cells would come to live. [4]

This is very similar to Conway's Game of Life, with the exception of having the same configuration but having rule parameters that could be changed easily. This meant that a large amount of designs could be iterated, the downside to this method was that most designs it produced were symmetrical. [4]

Architecture has to keep its surroundings in mind, the CA for now was just mindlessly mimicking and continuing to run without any constraints to make it resemble an architectural design. By adding a topographic model that was also constructed from cells it forced the CA to generate in a new way that took its surroundings into context. The new rule that was added told the CA that cells growing beyond parameter X along the Z axis would die depending on how its neighbours were configured. With this came the issue of how the architectures are supposed to interpret the results that are given to them, one answer is to give certain cells based on the amount of alive neighbours a unique colour as to distinguish between private and public spaces in our design. Further spaces were defined with new rules and new colours to make the design even easier to interpret. [4]

This led to architects understanding that the architects had to use their interpretation to give meaning to the various spaces that followed out of the CA. They also had to translate the given design to more logical forms for the various spaces the cells were located at, since the CA only produced square like figures and spaces. This led to the CA becoming more of a tool to the architects as to guide them and help them in exploring options rather than it being a mean to all ends that would solve every single one of their issues. [4]

This led to the discussion on how to use the CA's to their utmost potential and how we had to go about using them. One of the main ideas was to be able to stop the CA through its process, look at what it generated and having it start over while the previously generated figure would be kept at a distance. Doing this multiple times and mixing and matching the different figures would lead to better end results and promoted user interactivity rather than having the CA do its own thing without any supervision. This also helped mitigate the symmetric design issue that arose from having the CA run on its own. [4]

The way the CA use in architecture is tackled in this paper is a unique one that gives us new perspective into the use of CA's and how we could interpret them and play around with them giving us countless of final possibilities instead of the ones enforced by the rules themselves.

As far as we are concerned this revolutionary way of using CA's was definitely successful. Using a self-replicant algorithm to produce new patterns and interpreting them gives architects an infinite amount of possibilities in which they can use CA generated patterns to help them in designing new buildings.

The setups used in the paper pretty much covered all there was. The environmental context was taken into account by using cell based topography, and the designs themselves were translated to areas by defining them based on the cells in their vicinity. Lastly the CA was used in a unique manner to get interesting results that wouldn't necessarily be replicated by other architects.

9 Conclusion

Through constructing this GA, we have learned the basics of creating a GA for many kinds of tasks and learned about the versatility of this way of approaching problems that might be difficult to optimize. Even though this problem is relatively simple, we can infer that through the construction of several small, elegant functions and making use of principles similar to natural selection, we

can create agents that behave in complex ways or optimize difficult problems, without having to code everything by hand.

We've also learned that finding the optimal functions to modify your population with can be quite challenging, since there rarely seems to be just one correct answer. On top of this challenge, computing many generations of agents/strings can be very computationally expensive. This problem scales exponentially with the complexity of the genetic data, so this is to be kept in mind.

References

1. Bookstein, A., Kulyukin, V.A., Raita, T.: Generalized hamming distance. *Information Retrieval* **5**(4), 353–375 (2002)
2. Chicano, F., Whitley, D., Alba, E.: Exact computation of the expectation surfaces for uniform crossover along with bit-flip mutation. *Theoretical Computer Science* **545**, 76–93 (2014)
3. Deaven, D.M., Ho, K.M.: Molecular geometry optimization with a genetic algorithm. *Physical Review Letters* **75**(2), 288–291 (1995). <https://doi.org/10.1103/physrevlett.75.288>
4. Herr, C.M., Ford, R.C.: Cellular automata in architectural design: From generic systems to specific design tools. *Automation in Construction* **72**, 39–45 (2016). <https://doi.org/10.1016/j.autcon.2016.07.005>
5. Hirschberg, D.: Serial computations of levenshtein distances (1997)
6. Poli, R.: On fitness proportionate selection and the schema theorem in the presence of stochastic effects (2000)
7. Rabino, G.A., Laghi, A.: Urban cellular automata: the inverse problem. In: *International Conference on Cellular Automata*. pp. 349–356. Springer (2002)
8. Razali, N.M., Geraghty, J., et al.: Genetic algorithm performance with different selection strategies in solving tsp. In: *Proceedings of the world congress on engineering*. vol. 2, pp. 1–6. International Association of Engineers Hong Kong (2011)
9. Sarmady, S.: An investigation on genetic algorithm parameters. *School of Computer Science, Universiti Sains Malaysia* **126** (2007)
10. Soon, G.K., Guan, T.T., On, C.K., Alfred, R., Anthony, P.: A comparison on the performance of crossover techniques in video game. In: *2013 IEEE international conference on control system, computing and engineering*. pp. 493–498. IEEE (2013)
11. Wolfram, S.: *Cellular automata and complexity: collected papers*. CRC Press (2018)